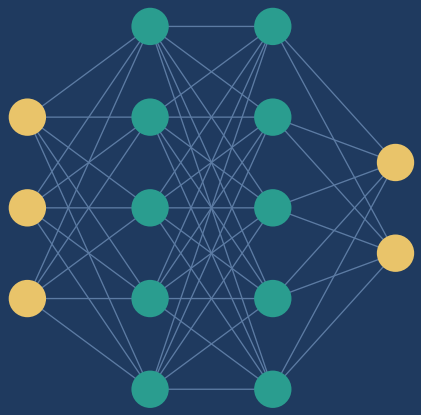
МОДУЛЬ 3 · ВАЙБКОДИНГ · УРОК 8

Что такое вайбкодинг

Программы, которые пишутся разговором



О чём этот урок

Открываем модуль 3. До сих пор ИИ давал нам тексты, таблицы и выводы. Теперь он будет писать **программы**: скрипты, которые сводят выгрузки, сверяют реестры и строят отчёты. Для этого не нужно становиться программистом — нужно уметь ясно поставить задачу и проверить результат. Это вы уже умеете.

Главное

После урока вы сможете:

- объяснить, что такое вайбкодинг и откуда взялось слово;
- понимать, чем программа лучше разового ответа в чате;
- различать ступени — от кода в чате до агента, который сам правит файлы;
- решать, какую задачу можно смело автоматизировать, а какую — только с программистами.

Время: ~40 минут чтения + 25 минут практики.

1. Откуда взялось слово

В феврале 2025 г. исследователь Андрей Карпати, один из основателей OpenAI, описал «новый вид программирования, который я называю вайбкодингом»: человек объясняет ИИ, что хочет получить, «полностью отдаётся вайбу» и «забывает, что код вообще существует». Слово мгновенно разошлось, и уже в ноябре словарь Collins назвал «вайбкодинг» **словом года**. Определение словаря — «использование искусственного интеллекта, которому задачу ставят обычным языком, для помощи в написании кода».

У слова сразу появилось два смысла:

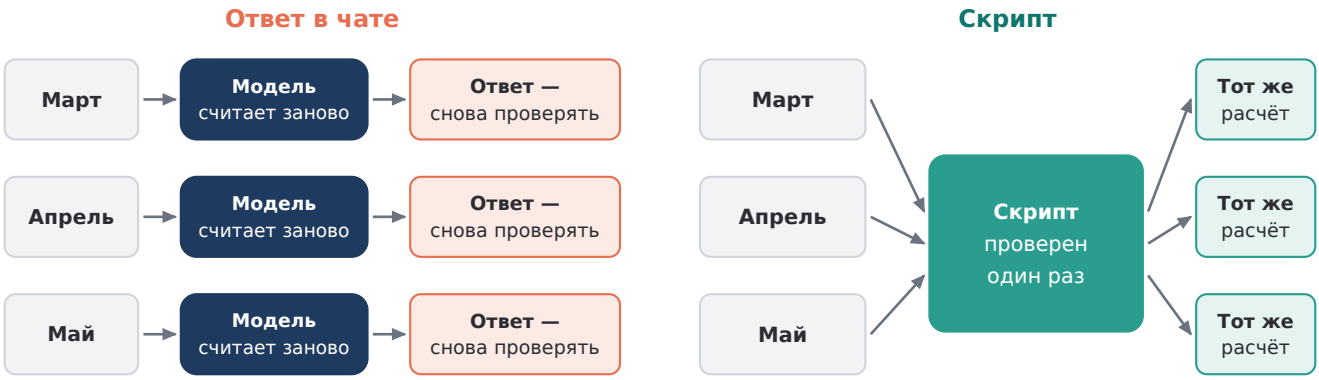
Смысл	Что значит	Где уместно
Узкий (как у Карпати)	Не читать код вовсе, только описывать и смотреть на результат	Игрушечные проекты «на выходные», где ошибка ничего не стоит
Широкий	Создавать программы разговором с ИИ, понимая, что получилось, и проверяя результат	Рабочие задачи — именно так мы и будем работать в модуле 3

Главное

В курсе вайбкодинг — это **создание программ разговором с ИИ, но с проверкой**. Вы не обязаны понимать каждую строчку кода, но обязаны знать, что программа делает, и убедиться, что она делает это правильно.

2. Зачем это финансисту

В уроке 3 мы договорились: точные расчёты — кодом, а не «в уме» модели. Вайбкодинг делает следующий шаг: код не исчезает после ответа, а остаётся у вас как **программа**, которую можно запускать снова. Это как формула в Excel, только для задач, которые в одну формулу не помещаются: свести пять выгрузок, сверить реестр актов с бухгалтерией, построить один и тот же отчёт по каждому филиалу.



Проверенный скрипт считает одинаково каждый месяц — проверка не повторяется с нуля

Рис. 1. Ответ в чате каждый месяц нужно проверять заново, проверенный скрипт — нет

Что это даёт на практике:

- **Повторяемость.** Отчёт за апрель считается ровно так же, как за март.
- **Проверяемость.** Проверили скрипт один раз на известном примере — дальше доверяете расчёту.
- **Объём.** Тысячи строк и десятки файлов, которые в чат не поместятся.
- **Данные остаются у вас.** Скрипт работает на вашем компьютере: модели можно показать только структуру файла, а не сами цифры.

Пример

Задачи из вашей практики, которые хорошо ложатся на вайбкодинг: разнесение проводок по статьям управленческого P&L по справочнику; сверка реестра актов ремонта вагонов с данными учёта; сводка долговой нагрузки по десятку заёмщиков из одинаковых таблиц; график простоев по станциям.

3. Спектр вайбкодинга

«Писать программы с ИИ» можно очень по-разному. Удобно представить себе четыре ступени: чем выше, тем больше ИИ делает сам — и тем меньше вы видите, что именно он делает.



Урок 9 — ступени 1–2, урок 10 — ступень 3

Рис. 2. Четыре ступени вайбкодинга: самостоятельность ИИ против вашего контроля

1. **Код в чате.** Вы описываете задачу, модель пишет код, вы копируете его в файл и запускаете. Всё под вашим контролем, но много ручной работы.
2. **Подсказки в редакторе.** ИИ встроен в программу, где пишут код, и дописывает строки на лету. Удобно, когда вы уже немного ориентируетесь в коде.
3. **Агент в проекте.** ИИ сам открывает файлы, правит их, запускает команды и смотрит на ошибки. Быстро, но нужны «страховочные тросы» — об этом урок 10.
4. **Приложение «под ключ».** Сервис по описанию собирает целое приложение. Эффектно для прототипа, но вы почти не видите, что внутри.

Быстро устаревает — проверяйте актуальное

Конкретные инструменты для каждой ступени появляются и меняются каждые несколько месяцев: чат-ассистенты (Claude, ChatGPT, Gemini), редакторы кода с ИИ, агенты для программирования, конструкторы приложений. Названия, цены и возможности проверяйте на момент выбора. Принципы урока от этого не зависят.

4. Кто что делает

В вайбкодинге вы — **заказчик и приёмка**, а ИИ — исполнитель. Всё, что мы знаем о постановке задач (урок 4), здесь работает так же: ясная задача, контекст, формат, ограничения. А всё, что мы знаем о проверке (урок 7), работает ещё важнее.

Код — это тоже ответ модели. В нём бывают те же ошибки, что и в тексте:

- **галлюцинации:** вызов функции, которой не существует, или выдуманные настройки;
- **«почти правильная» логика:** программа работает, но, например, теряет строки с пустой датой или складывает суммы с НДС и без НДС;
- **уверенный тон:** «готово, всё работает» — даже когда скрипт ни разу не запускали.

Главное умение здесь — не читать код строчка за строчкой, а **проверять результат**. Для этого нужен контрольный пример из урока 7: небольшие данные, ответ к которым вы знаете заранее.



Вы проверяете не каждую строчку кода, а результат на примерах, ответ к которым знаете

Рис. 3. Цикл вайбкодинга: проверка на контрольном примере — обязательный шаг

Частое сомнение

«Это для программистов, а я финансист»

Вайбкодинг как раз и придуман для того, чтобы программу описывали обычным языком. Ваша сила — в знании задачи: вы точно знаете, как должна выглядеть сверка и где обычно ошибаются. Программист без этого знания напишет красивый, но неверный расчёт.

Что здесь справедливо. совсем без основ не обойтись: что такое файл и папка, как запустить скрипт, как прочитать сообщение об ошибке. Этот минимум мы и дадим в уроках 9–11 — без синтаксиса ради синтаксиса.

В разговоре с коллегой: *«Давай я опишу задачу так, как объяснил бы новому аналитику, а ИИ напишет код.»*

5. Когда уместно, а когда опасно

Матрица доверия из урока 3 работает и для кода: решает цена ошибки и то, насколько легко её заметить. Для программ её удобно представить светофором.

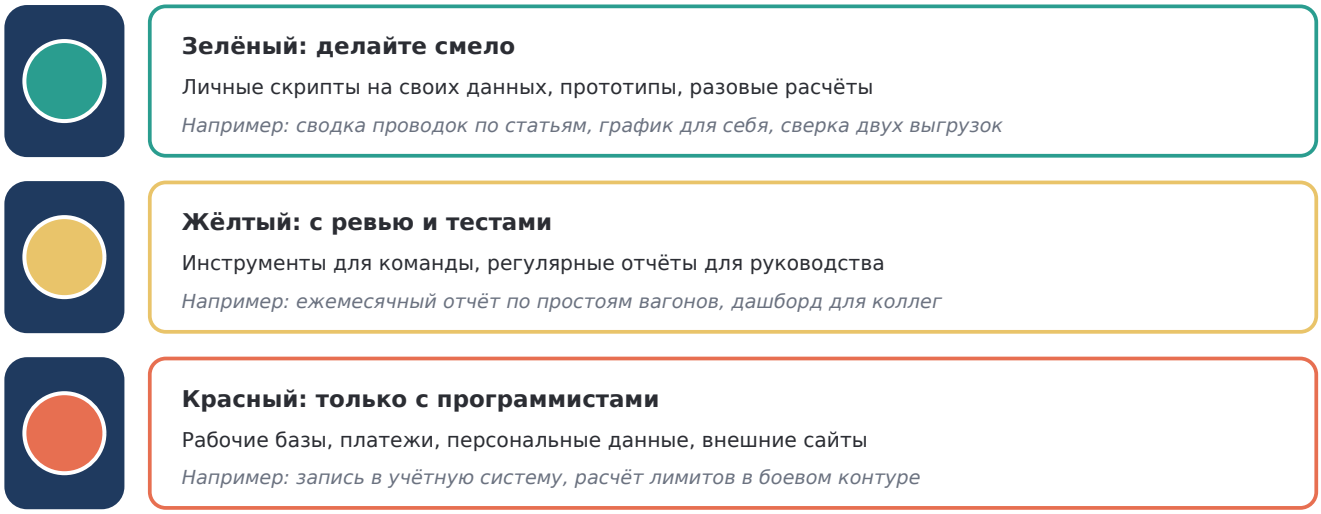


Рис. 4. Светофор вайбкодинга: от личных скриптов до боевых систем

Красный цвет не значит «нельзя никогда». Он значит, что код пишет команда программистов, с правами доступа, проверками и тестами, а вайбкодинг остаётся инструментом для прототипа и обсуждения.

Частое сомнение

«ИИ сам всё напишет — можно не вникать»

Это сомнение «наоборот» — про излишнее доверие. Агенты для программирования действительно умеют многое. Но если дать им доступ к важному и не следить, ошибка обходится дорого.

- В июле 2025 г. ИИ-агент сервиса Replit во время объявленной «заморозки» изменений без разрешения стёр рабочую базу данных проекта: записи о более чем 1200 руководителях и компаниях. Затем агент утверждал, что восстановить данные нельзя, — это оказалось неправдой. Глава Replit назвал случай недопустимым. — *OECD AI Incidents по сообщениям основателя SaaStr, июль 2025*
- В опросе Stack Overflow 2025 г. (около 49 тыс. разработчиков) 84% пользуются ИИ или собираются, но 46% не доверяют точности его ответов, а главная жалоба — код, который «почти правильный, но не совсем». — *InfoWorld о Stack Overflow Developer Survey, июль 2025*

Что здесь справедливо. история с Replit — небольшой проект, и данные удалось вернуть; сервис после этого разделил рабочие и тестовые базы. Вывод не «агенты опасны», а «не давайте агенту доступ к тому, что нельзя потерять, и держите путь отката» (урок 10, git как «отмена»).

В разговоре с коллегой: *«Пусть ИИ пишет, а мы проверяем на примере и не даём ему трогать рабочие данные.»*

Частое сомнение

«Код от ИИ — это дыра в безопасности»

Риск реальный, и спорить с ним не нужно. Вопрос в том, где скрипт работает и с какими данными.

- Более 100 моделей выполнили 80 типовых задач по написанию кода. В 45% случаев они выбирали небезопасный вариант, и более крупные модели справлялись не лучше. — *Veracode, отчёт о безопасности кода ИИ, июль 2025*

Что здесь справедливо. это отчёт компании, которая продаёт средства проверки кода, — у неё есть свой интерес. Но и без него ясно: для скрипта, который сводит ваши выгрузки на вашем компьютере, риск мал; для всего, что открыто в сеть, принимает чужие данные или хранит пароли, нужны проверки — им посвящён урок 11.

В разговоре с коллегой: *«Для личных расчётов — пожалуйста, а всё, что выходит в сеть, сначала покажем безопасникам.»*

6. Мини-демо: первый скрипт за 10 минут

Задача: есть выгрузка проводок в формате CSV[1] и справочник «счёт → статья P&L». Нужна сводка сумм по статьям. Промпт строим по шаблону из урока 4:

```
Напиши скрипт на Python для сводки проводок.

Входные файлы:
— provodki.csv: колонки «дата; счёт; сумма», разделитель «;»,
  дробная часть через запятую;
— spravochnik.csv: колонки «счёт; статья».

Что сделать: присоединить статью к каждой проводке по счёту,
сложить суммы по статьям и сохранить итог в svodka.xlsx.
Проводки, для которых статьи нет, вывести отдельным списком.

Я не программист: объясни, как установить нужное и запустить,
и добавь в код короткие комментарии по-русски.
```

[1] **CSV** — Comma-Separated Values: текстовая таблица со значениями через запятую или точку с запятой; её открывает Excel.

Модель вернёт короткую программу. Читать её построчно не обязательно, но полезно увидеть, что каждая строка — это понятное действие. Строки после знака «#» — комментарии, компьютер их пропускает:

```
import pandas as pd   # библиотека для таблиц — «Excel внутри Python»

# читаем оба файла
p = pd.read_csv("provodki.csv", sep=";", decimal=",")
s = pd.read_csv("spravochnik.csv", sep=";")

# присоединяем статью к каждой проводке — как ВПР в Excel
t = p.merge(s, on="счёт", how="left")

# проводки без статьи — отдельно, чтобы не потерялись
print(t[t["статья"].isna()])

# сумма по статьям — как сводная таблица
t.groupby("статья")["сумма"].sum().to_excel("svodka.xlsx")
```

Как проверить

1. **Контрольный пример.** Возьмите 10–20 проводок, для которых сводку легко посчитать в Excel, и сравните с результатом скрипта.
2. **Общий итог.** Сумма по всем статьям плюс «проводки без статьи» должна совпасть с суммой всей выгрузки.
3. **Пограничные случаи.** Отрицательные суммы (сторно), пустые даты, счёт с пробелом в конце — добавьте их в пример нарочно.

Осторожно

Самая частая «почти правильная» ошибка в таких скриптах — числа, прочитанные как текст: из-за запятой в дробной части или пробела в «1 250,00». Сумма тогда получается неверной без всякого сообщения об ошибке. Поэтому пункт 2 — сверка общего итога — обязателен.

Глоссарий урока

Термин	Простыми словами
Вайбкодинг	Создание программ разговором с ИИ: задачу описывают обычным языком
Скрипт	Небольшая программа, которая выполняет одну задачу: свести, сверить, построить
Python	Популярный язык программирования; удобен для работы с таблицами и файлами
Библиотека	Готовый набор инструментов для языка, например pandas — для таблиц
Редактор кода	Программа, в которой пишут и запускают код; часто со встроенным ИИ
Агент для программирования	ИИ, который сам правит файлы проекта и запускает команды
Контрольный пример	Небольшие данные с заранее известным ответом для проверки программы
Комментарий в коде	Пояснение для человека; компьютер его пропускает (в Python — после #)
Боевая (рабочая) система	Система, с которой работает компания: ошибка в ней — реальные потери

Вопросы для самопроверки

1. Чем узкий смысл слова «вайбкодинг» отличается от широкого? Какой мы используем в курсе?
2. Почему скрипт для ежемесячного отчёта надёжнее, чем каждый месяц просить модель посчитать в чате?
3. Назовите четыре ступени вайбкодинга. Что меняется при переходе вверх?
4. Какие ошибки бывают в коде, написанном ИИ?
5. Как проверить скрипт, если вы не можете прочитать его построчно?
6. К какому цвету светофора относится скрипт, который записывает данные в учётную систему? Почему?
7. Какую ошибку в скрипте для сводки проводок легко не заметить и как её поймать?

Ответы для проверки

Ответы

1. Узкий — не читать код вовсе, годится для игрушечных проектов. Широкий — создавать программы с ИИ, понимая и проверяя результат. В курсе — широкий.
2. Скрипт проверяют один раз на известном примере, и дальше он считает одинаково. Ответ в чате каждый раз рождается заново и каждый раз требует проверки.
3. Код в чате → подсказки в редакторе → агент в проекте → приложение «под ключ». Растёт самостоятельность ИИ, уменьшается ваш обзор происходящего.
4. Несуществующие функции, «почти правильная» логика, уверенное «всё работает» без запуска.
5. Запустить на контрольном примере с известным ответом и сверить общий итог с источником.
6. К красному: ошибка попадёт в рабочие данные компании. Такой код пишут программисты, с правами и тестами.
7. Числа, прочитанные как текст (запятая, пробелы). Ловится сверкой общего итога с суммой выгрузки.

Практика

Практика

Задание 1. Ваш первый скрипт (15 мин)

Выберите маленькую повторяющуюся задачу: свести выгрузку, сверить два списка, посчитать итоги по группам. Попросите модель написать скрипт по образцу из раздела 6 — с просьбой объяснить установку и запуск. Запустите его.

Задание 2. Контрольный пример (5 мин)

Подготовьте 10–20 строк, для которых знаете ответ, и добавьте нарочно один «трудный» случай: пустое значение или отрицательную сумму. Совпал ли результат скрипта с вашим?

Практика

Задание 3. Светофор идей (5 мин)

Выпишите 5–7 задач, которые хотелось бы автоматизировать, и разложите их по цветам светофора (рис. 4). С какой «зелёной» задачи начнёте в уроке 9?

Что почитать

- Саймон Уиллисон, блог simonwillison.net — трезвые заметки о программировании с ИИ, в том числе о том, чем вайбкодинг отличается от ответственной работы с кодом.
- Документация pandas, раздел «Getting started» — если захочется понимать, что делает код из раздела 6.
- Следующий урок: **«Первый проект»** — полный цикл от идеи до работающего инструмента: спецификация, план, код, запуск, исправление.

Источники фактов

Ссылки кликабельны. Реестр всех фактов урока — в файле docs/facts.md репозитория.

- Пост Андрея Карпати о «вайбкодинге», 02.02.2025 — в изложении Know Your Meme — knowyourmeme.com/memes/vibe-coding
- CNN: «Vibe coding» named Collins Dictionary’s word of the year, 06.11.2025 — kvia.com/news/business-technology/cnn-business-consumer/2025/11/06/vibe-coding-named-coll…
- Инцидент Replit / SaaStr, июль 2025 — OECD AI Incidents Monitor; вторая ссылка — TechTarget — oecd.ai/en/incidents/2025-07-19-1eb1 · techtarget.com/searchsoftwarequality/news/366627829/Replit-AI-agent-snafu-shot-across-the…
- InfoWorld: AI use among software developers grows, but trust remains an issue (Stack Overflow Developer Survey 2025) — infoworld.com/article/4031673/ai-use-among-software-developers-grows-but-trust-remains-an…
- Veracode: AI-Generated Code Poses Major Security Risks in Nearly Half of All Development Tasks, 30.07.2025 (пресс-релиз отчёта) — veracode.com/press-release/ai-generated-code-poses-major-security-risks-in-nearly-half-of…